

LAB GUIDELINES

Lab 18 — Data at Rest (LUKS) and Data in Transit (TLS / SSH)

Maps to CompTIA Server+ objective **3.1 — Summarize data security concepts** (encryption paradigms: data at rest, data in transit; retention policies; UEFI/BIOS passwords; bootloader passwords; business impact — data value prioritisation, life-cycle management, cost of security vs. risk).

Run all commands on <https://killercode.com/playgrounds/scenario/ubuntu>

Required software (free, apt): cryptsetup, openssl, openssh-server

```
apt update && apt install -y cryptsetup openssl openssh-server
```

Step 1 — Encrypt a volume at rest with LUKS

```
truncate -s 200M /srv/secret.img
losetup /dev/loop20 /srv/secret.img
cryptsetup luksFormat /dev/loop20 --batch-mode --key-file=<(echo -n 'LabPass!1')
cryptsetup luksOpen /dev/loop20 secret --key-file=<(echo -n 'LabPass!1')
mkfs.ext4 /dev/mapper/secret
mkdir -p /mnt/secret && mount /dev/mapper/secret /mnt/secret
echo "financials Q1" > /mnt/secret/data.txt
```

Now close it and prove the raw bytes are unreadable:

```
umount /mnt/secret
cryptsetup luksClose secret
strings /srv/secret.img | grep financials || echo "no plaintext visible – at-rest encryption works!"
```

Re-open with the wrong passphrase to confirm rejection:

```
cryptsetup luksOpen /dev/loop20 secret --key-file=<(echo -n 'wrong') || echo "rejected (as expected)"
```

Step 2 — Cipher and header inspection

```
cryptsetup luksOpen /dev/loop20 secret --key-file=<(echo -n 'LabPass!1')
cryptsetup status secret
cryptsetup luksDump /dev/loop20 | head -30
```

Default modern cipher: **aes-xts-plain64** with **SHA-256** keyslot KDF (argon2id on newer versions).

Step 3 — Data in transit: TLS

Generate a self-signed cert and serve over HTTPS:

```
openssl req -x509 -newkey rsa:2048 -nodes \
  -keyout /etc/ssl/private/lab.key -out /etc/ssl/certs/lab.crt \
  -subj "/CN=lab.local" -days 365 2>/dev/null
chmod 600 /etc/ssl/private/lab.key

apt install -y nginx
cat > /etc/nginx/sites-available/tls <<'EOF'
server {
    listen 443 ssl;
    server_name lab.local;
    ssl_certificate /etc/ssl/certs/lab.crt;
    ssl_certificate_key /etc/ssl/private/lab.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
    root /var/www/html;
    index index.html;
}
EOF
ln -sf /etc/nginx/sites-available/tls /etc/nginx/sites-enabled/
```

```
nginx -t && systemctl reload nginx
curl -kIv https://127.0.0.1/ 2>&1 | grep -E 'SSL|TLS|HTTP/' | head
```

Step 4 — Verify the negotiated cipher and protocol

```
echo | openssl s_client -connect 127.0.0.1:443 -servername lab.local 2>/dev/null \
| grep -E "Protocol|Cipher\s+:"
```

Production targets: TLS 1.2 minimum, TLS 1.3 preferred, only AEAD ciphers (GCM, ChaCha20-Poly1305), HSTS in headers.

Step 5 — Data in transit: SSH

```
ssh -V
ssh -Q cipher | head
ssh -Q kex | head
ssh -Q mac | head
```

Hardening edits in `/etc/ssh/sshd_config`:

```
Protocol 2
PermitRootLogin prohibit-password
PasswordAuthentication no
KexAlgorithms curve25519-sha256
Ciphers chacha20-poly1305@openssh.com,aes256-gcm@openssh.com
MACs hmac-sha2-256-etm@openssh.com
```

`systemctl restart ssh` after edits.

Step 6 — UEFI/BIOS and bootloader passwords

Server+ 3.1 calls these out explicitly. They can't be set inside the OS — they're set during the **POST/boot phase**:

Where	How
BIOS/UEFI password	Vendor setup screen (Del / F2 / F10) → Security → Set Supervisor Password
Boot order lock	Same menu — prevent USB / PXE boot of a rogue OS
GRUB password	<code>`grub-mkpasswd-pbkdf2`</code> , add <code>`password_pbkdf2 root <hash>`</code> to <code>/etc/grub.d/40_custom</code> , <code>`update-grub`</code>

Demonstration of the GRUB password format:

```
echo -e "LabPass!1\nLabPass!1" | grub-mkpasswd-pbkdf2 2>/dev/null | tail -1
```

Step 7 — Data retention policy and business impact

Server+ 3.1 ties retention + lifecycle directly to the BIA from Lab 17. A simple policy template:

Classification	Encryption	Retention	Destruction
Public	Optional	As needed	Standard delete
Internal	TLS in transit	3 years	Overwrite (Lab 25)
Confidential	LUKS at rest + TLS	7 years	Shred / degauss
Restricted	HW-encrypted disk + TLS 1.3 + MFA	7+ years, legal hold	Physical destruction + certificate

Step 8 — Storage location: off-site vs. on-site

Encryption keys must never live on the same volume as their ciphertext. Off-site vs. on-site backups (Lab 26) extend the same rule: 3-2-1 means **3 copies on 2 media with 1 off-site**.

Free online tools

- ****SSL Labs server test**** — <https://www.ssllabs.com/ssltest/>
- ****Mozilla SSL config generator**** — <https://ssl-config.mozilla.org/>
- ****cryptsetup FAQ**** — <https://gitlab.com/cryptsetup/cryptsetup/-/wikis/FrequentlyAskedQuestions>
- ****NIST SP 800-57 key management**** — <https://csrc.nist.gov/publications/sp>

What you learned

- Full LUKS at-rest encryption lifecycle: format, open, mount, close, verify ciphertext.
- Self-signed TLS for in-transit protection and inspection of the negotiated cipher.
- SSH protocol hardening lines you can paste into `sshd_config`.
- BIOS/UEFI and GRUB password roles — and that they live outside the OS.
- Retention policy mapped to data classification.
- Why keys and ciphertext must never share storage.